DEVELOPER PORTFOLIO / 2026.09

조윤호

인프라와 운영 도구 개발

개인 서버에서 서비스를 운영하며, 배포와 VM 관리에 필요한 도구를 직접 만들고 사용하고 있습니다. 운영 중 발견한 불편을 기능으로 옮기고, 실제 환경에서 확인하며 개선합니다.

Homelab · 개인 서비스 운영 환경 02

01 **K-Le-PaaS** 03–05
자연어 요청과 모니터링을 연결한 팀 프로젝트
프로젝트 개요 · Architecture · 구현과 경험

02 **Heimdall** 06–08
개인 서비스를 배포하고 관리하는 도구
프로젝트 개요 · Architecture · In Use

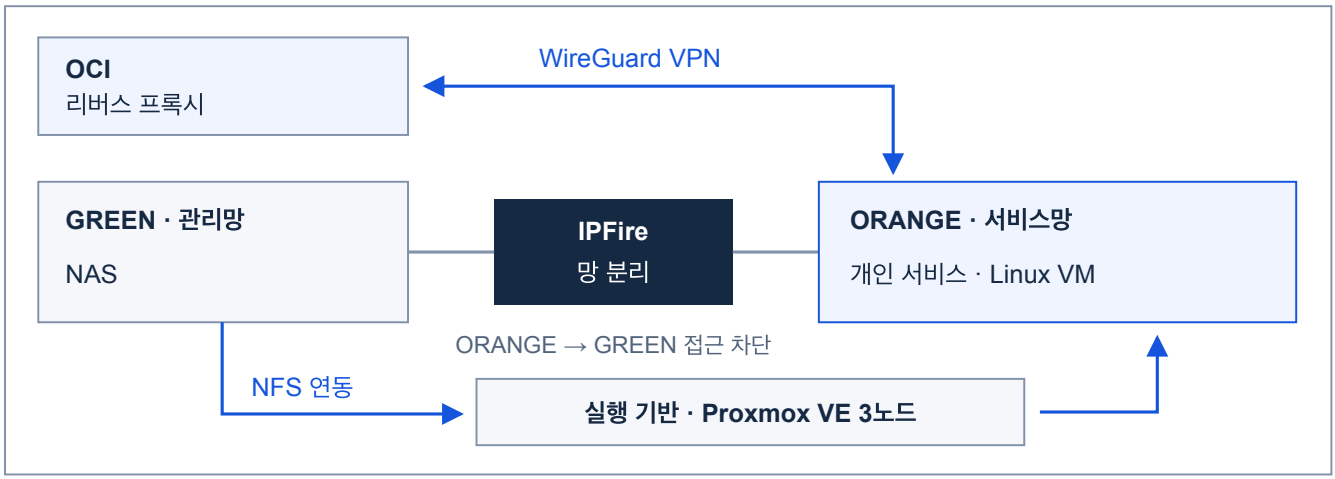
03 **Gjallar** 09–11
홈랩의 VM을 만들고 관리하는 도구
프로젝트 개요 · Architecture · In Use

code.penguin.yoon@gmail.com
github.com/CodingPenguin-yoon · yoonman.page

Homelab

개인 서비스 운영 환경

게임 서버를 24시간 사용하기 위해 시작한 홈서버를 여러 개인 서비스를 운영하는 환경으로 확장했습니다. Proxmox 3노드와 Linux VM을 사용하며, 네트워크와 스토리지를 직접 구성했습니다.



개인 홈랩의 논리 구성입니다. 장비별 물리 배치와 세부 라우팅은 생략했습니다.

외부 접속

OCI는 리버스 프록시로 사용하고 실제 서비스는 홈랩에 두었습니다. OCI와 서비스망은 WireGuard VPN으로 연결했습니다.

네트워크 분리

IPFire로 GREEN 관리망과 ORANGE 서비스망을 나눴습니다. 외부에 서비스를 제공하면서 ORANGE에서 GREEN으로의 접근은 차단했습니다.

실행 환경과 스토리지

Proxmox에서 Linux VM을 운영하고, GREEN의 NAS를 NFS로 연동했습니다. 별도로 Linux VM에 하드디스크를 연결해 직접 마운트하는 구성도 사용했습니다.

개발과 검증

개인 서비스를 운영하면서 직접 만든 도구의 기능도 이 환경에서 확인합니다. 시험 VM의 생성·설정 변경과 개인 서비스 배포를 확인하고, 실제 접속 환경에서 발견한 문제를 고치며 개발을 이어 가고 있습니다.

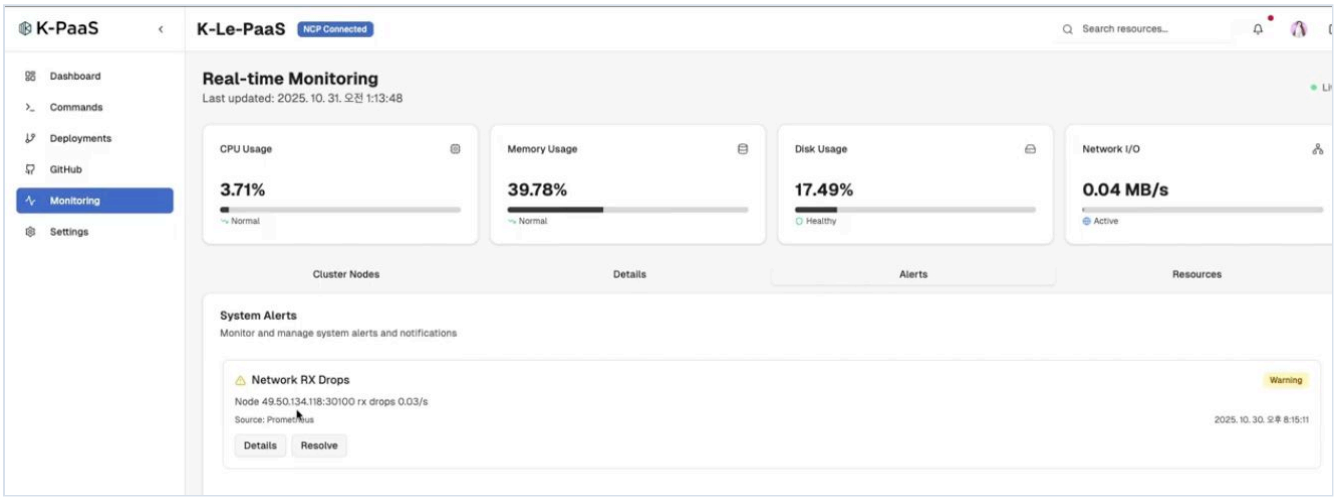
Proxmox VE · Linux · IPFire · WireGuard · OCI · NAS/NFS

01 K-Le-PaaS

자연어 기반 Kubernetes 운영 플랫폼

2025.09–2025.12 · 2인 팀
담당: 자연어 제어 · 인프라 모니터링

여러 도구와 명령을 익혀야 하는 운영 부담을 줄이기 위해 친구와 만든 플랫폼입니다. 자연어로 운영 작업을 요청하고, Git 연동 배포와 자원 상태를 한 화면에서 확인하도록 구성했습니다.



2025년 팀 시연 화면. CPU·메모리 등 자원 사용량과 Prometheus 경고를 함께 보여줍니다.

운영 작업

자연어 제어

재시작·상태 조회·파드 수 조절을 문장으로 요청하고 처리 결과를 확인합니다.

서비스 배포

Git 연동

코드를 변경하면 빌드·배포가 시작되고, 웹에서 진행 상황과 결과를 확인합니다.

상태 확인

모니터링

CPU·메모리·디스크·네트워크 사용량과 Prometheus 경고를 확인합니다.

내 역할 자연어 의도·대상 변환, 허용 작업 연결, Prometheus 설치·수집 대상 구성, 지표 조회 API와 화면 연결을 맡았습니다.

Git 연동 빌드·배포는 팀 결과입니다. 롤백과 Prometheus client의 초기 뼈대는 팀원이 맡았고, 별도 WebSocket 경로는 공동 구현입니다.

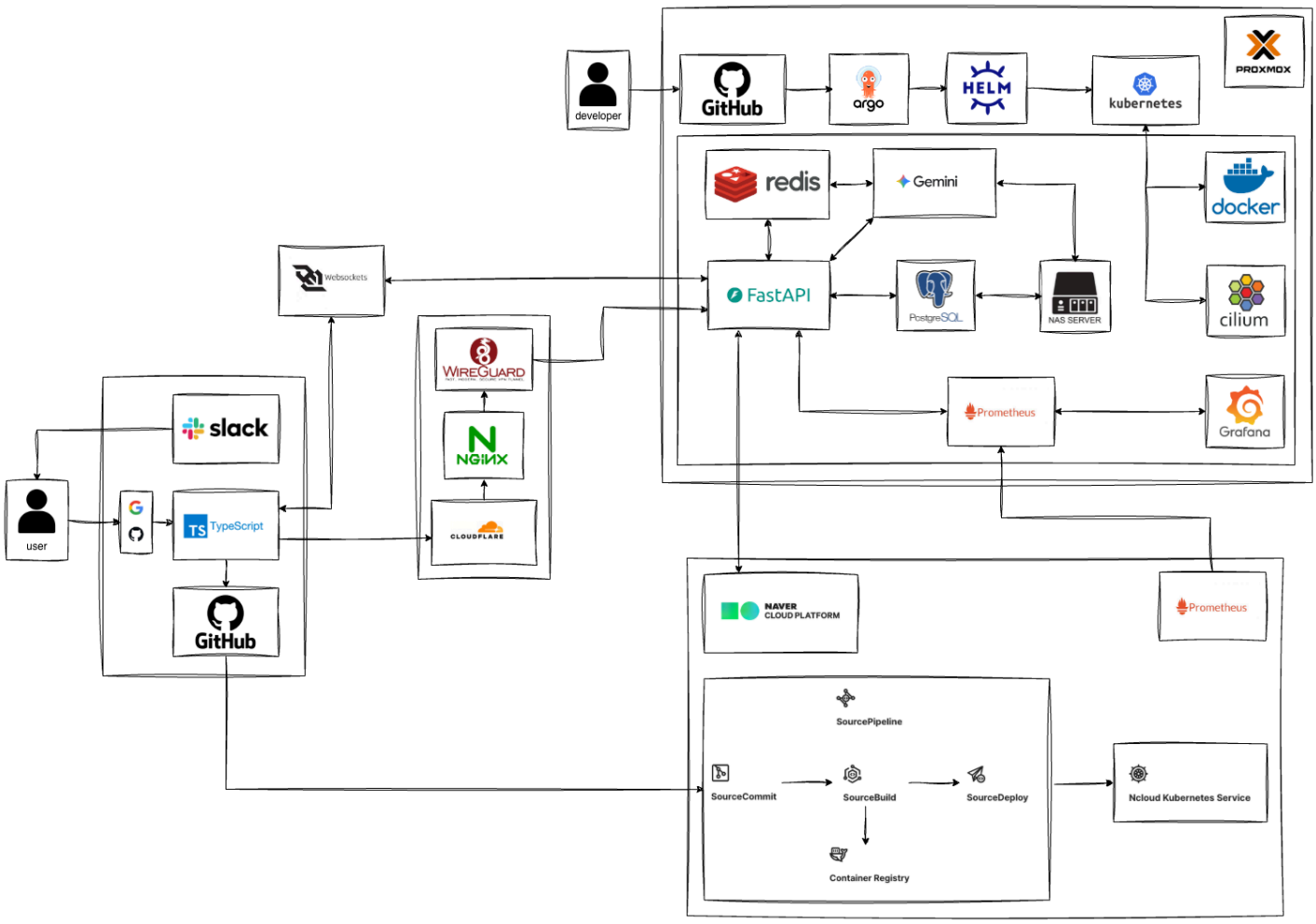
Python · FastAPI · Gemini · Kubernetes · Prometheus · React/TypeScript

Backend · Frontend · 팀 시연

Architecture

운영 제어와 모니터링

자연어 엔진과 Prometheus는 개인 온프레미스에, 서비스 배포 환경은 NCP/NKS에 두었습니다. 아래는 Git 연동 배포를 포함한 팀 프로젝트 전체 구조입니다.



저장소의 원본 이미지. 제 담당은 자연어 제어·지표 수집 및 조회·대시보드 연결입니다.

제어 요청

자연어에서 실행까지

자연어 엔진이 요청을 해석하면 백엔드가 입력 형식을 검증합니다. 이후 재시작·상태 조회·파드 수 조절의 실행 경로로 연결합니다.

운영 지표

수집에서 화면까지

제가 구성한 Prometheus에서 지표를 수집합니다. 조회 API로 지표와 경고를 제공하고 대시보드에 표시하도록 연결했습니다.

아키텍처 원본 · 팀 프로젝트 저장소

구현과 경험

자연어 제어와 모니터링 개발

자연어 요청 처리

자연어 요청에서 작업과 대상을 구분해 실행으로 연결하는 부분을 맡았습니다. Gemini의 해석 결과를 정해진 입력 형식으로 검증하고, 재시작·상태 조회·파드 수 조절 등 지원하는 작업의 실행 함수에 전달했습니다.

`K-Le-PaaS/test01` 재시작해줘와 같은 요청에서는 저장소 정보가 있는지 확인하고, 해당 Deployment를 조회한 뒤 Kubernetes API로 재시작을 요청합니다. 파드 수 조절은 NCP 매니페스트 변경 경로에 연결했습니다.

지표 수집과 API·화면 연결

제가 운영하는 Kubernetes에 Prometheus를 설치하고 수집 대상을 구성했습니다. 팀원이 먼저 만든 Prometheus client를 바탕으로 CPU·메모리·디스크·네트워크 조회 API를 구현했고, 이후 상세 지표와 알림 조회를 추가했습니다.

대시보드에는 REST API를 10초마다 조회해 자원 사용량과 경고를 갱신하도록 연결했습니다. 별도의 WebSocket 전달 경로는 팀원과 함께 구현했습니다.

운영 도구 개발의 출발점

K-Le-PaaS에서 자연어 요청을 실행으로 연결하고 운영 지표를 화면에 보여주는 기능을 만들며 SRE에 관심을 갖게 됐습니다. 이후 개인 서버에서 반복하던 서비스 배포와 VM 관리 작업을 줄이는 도구를 개발했습니다. 앱 배포는 Heimdall, VM 관리는 Gjallar가 맡도록 역할을 나눠 직접 사용하고 있습니다.

동작 확인

팀 시연에서 자연어 요청에 따른 파드 수 변경과 자원 지표·경고 표시를 확인했습니다. 자연어 제어 · 모니터링

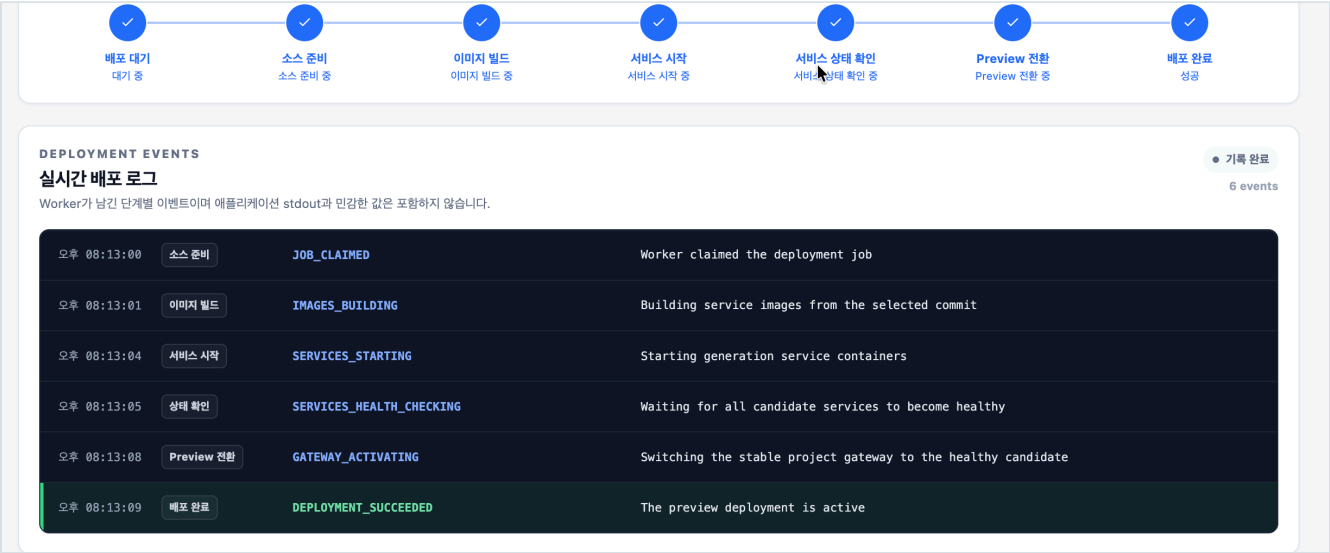
요청 처리 코드 · 지표 API #42 · 상세 지표·알림 #108 · 화면 연결 #22

02 Heimdall

개인 서비스 배포 도구

2026.08–현재 · 현재 구현 버전 / 초기 개발 2026.01
개인 프로젝트 · 설계·구현·배포·검증

홈페이지와 GitHub 트래커 같은 서비스를 추가할 때마다 VM과 실행 환경, 접속 경로를 준비했습니다. 반복되는 준비를 줄이기 위해 공개 GitHub 저장소와 설정을 등록하면 공용 서버에 앱을 배포하는 웹 도구를 만들었습니다.



기존 버전의 실제 배포 화면. 준비 상태 확인, Preview 전환과 완료 로그를 보여줍니다.

기존 버전 · 운영 중

개인 서비스 배포

홈페이지·GitHub 트래커 배포에 사용합니다. 저장소 등록부터 이미지 빌드·응답 확인·접속 경로 전환까지 연결하고, 웹에서 진행 상태와 로그를 확인합니다.

최신 구현 · 로컬 검증

배포와 DB 조회 확장

main의 최신·최근 commit을 선택해 HTTP 서비스와 상시 워커를 배포하고, 프로젝트 DB를 읽기 전용으로 조회합니다. 운영 서버 적용 전입니다.

내 역할 관리 화면·API·상태 저장과 Docker 배포 처리를 연결했습니다.

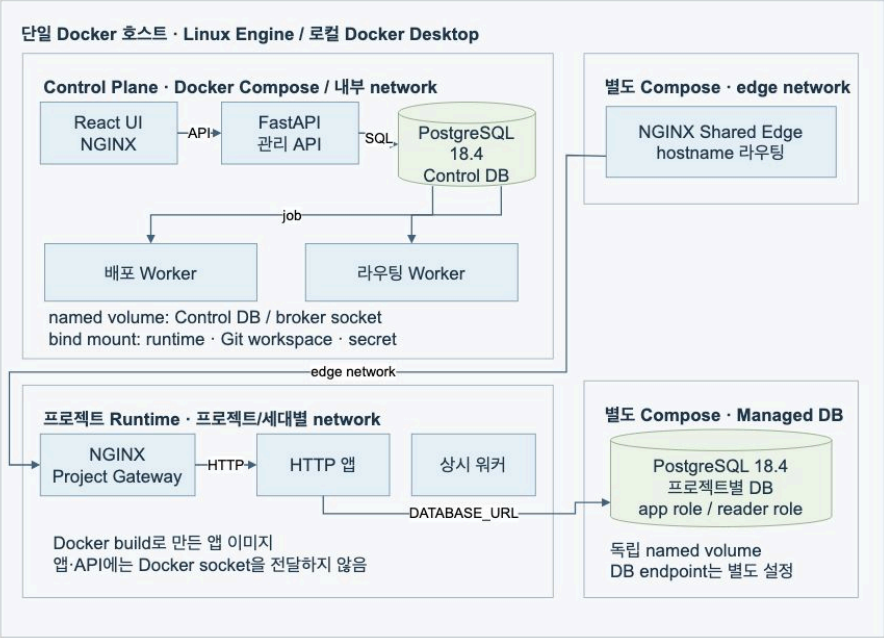
React · FastAPI · PostgreSQL · Docker · NGINX

코드·사용 가이드 · 배포 시연

Architecture

배포 관리와 서비스 실행

API는 배포 요청과 설정을 저장하고 Worker가 빌드·실행을 수행합니다. 배포된 서비스의 접속 경로와 앱 데이터는 관리 계층에서 분리했습니다.



최신 구현 · 로컬 검증(2026.09.26). 단일 Docker 호스트 구성으로 운영 서버 적용 전입니다.

배포 전환

준비 확인 후 새 버전 연결

요청 시점의 소스와 설정을 저장하고 새 컴포넌트의 준비 상태를 확인합니다. HTTP 서비스와 상시 워커의 확인 방식을 나누고, 교체 실패 시 이전 세대를 복구하도록 구성했습니다.

데이터 관리

설정과 앱 데이터 분리

배포 상태를 담는 Control DB와 프로젝트 DB를 분리했습니다. 앱에 DB 연결 정보를 전달하고, 관리 화면의 데이터 조회에는 별도의 읽기 전용 계정을 사용합니다.

배포 확인 개선

네트워크 변경 후 Preview 접속이 끊기는 문제를 재현해, 호스트 응답을 다시 확인한 뒤 활성화하도록 고쳤습니다. 실패 시 이전 Gateway를 복원합니다. 수정 기록

Gateway 재생성과 상시 워커 교체에는 중단 구간이 있어 무중단을 보장하지 않습니다.

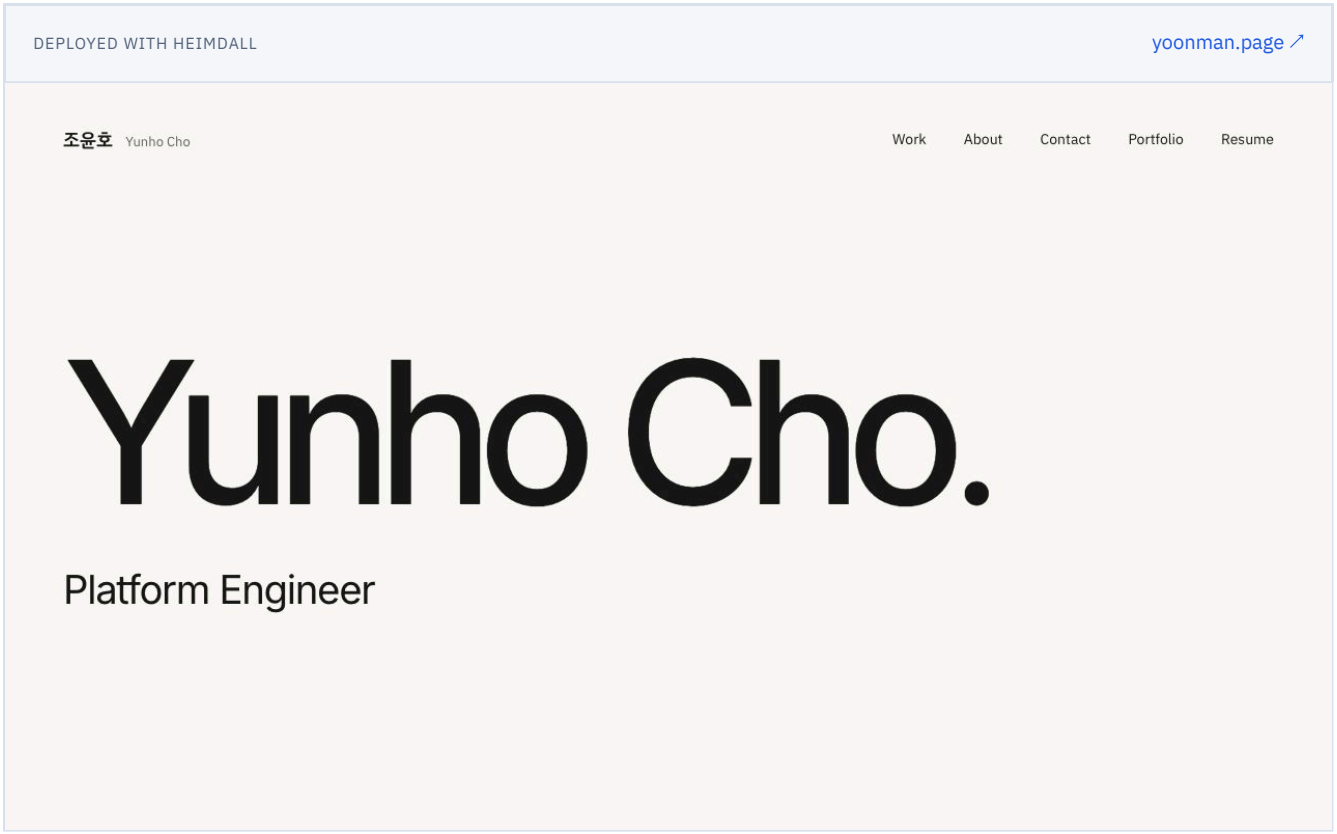
배포 구조 · 웹 DB 조회 구현

In Use

내 서비스를 올리는 배포 플랫폼

● **기존 버전 · 개인 서비스 운영** 단일 VM · Docker

Heimdall로 홈페이지와 GitHub 트래커 등 개인 서비스를 배포해 운영하고 있습니다. 배포에 필요한 설정과 진행 상황을 웹에서 관리합니다.



운영 중인 개인 홈페이지의 실제 접속 화면 · 2026.09.28

하나의 환경에 서비스 추가

준비된 공용 서버에 앱을 추가하는 방식으로 운영하고 있습니다. 서비스마다 VM을 새로 만들지 않고, 앱별 설정과 배포 과정을 Heimdall에서 관리합니다.

홈페이지 배포와 재배포

저장소 등록부터 이미지 빌드, 서비스 응답·접속 경로 확인과 Preview 전환까지 직접 사용했습니다. 홈페이지의 최초 배포와 이후 재배포를 확인했습니다.

HTTP·상시 워커 배포와 웹 DB 조회로 확장한 최신 구성은 운영 서버 적용 전입니다.

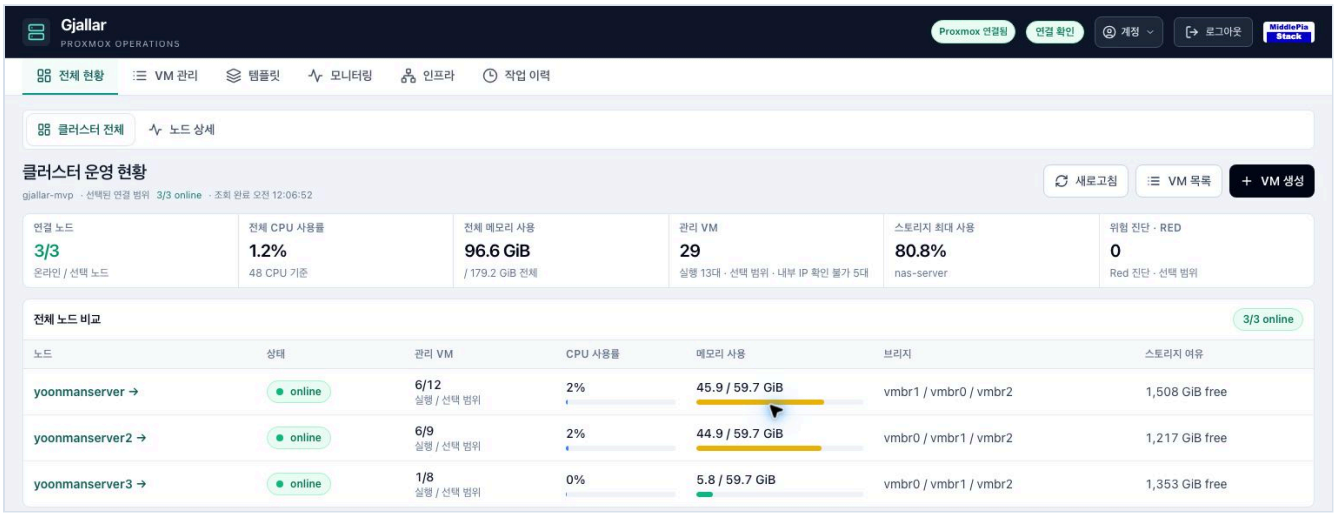
운영 홈페이지 · 배포 시연 · 프로젝트 저장소

03 Gjallar

Proxmox VM 관리 도구

2026.05–현재 · 개인 프로젝트 · 로컬에서 사용 중
설계·구현·배포·검증

홈랩에서 VM마다 사양과 IP를 다시 정했고, 주소를 수동으로 관리하다 충돌을 겪었습니다. 자주 쓰는 사양을 재사용하고, 생성 전에 실제 자원 조건을 확인할 수 있도록 관리 도구를 만들었습니다.



실제 홈랩 조회 화면(2026.09.27). 노드 3대와 VM 29개는 캡처 시점의 상태입니다.

01 / 조회

홈랩 현황 확인

노드·VM·스토리지 현황과 자원 사용량을 조회하고, 관리할 대상을 선택합니다.

02 / 생성

템플릿·사양 선택

기존 템플릿과 사양을 선택하고 생성 계획을 검토합니다. 사전 검사와 승인을 거쳐 실행합니다.

03 / 관리

설정·접속·이력

정지 VM의 지원 사양을 변경하고 실행 중 VM의 콘솔을 엽니다. 작업 결과와 이력도 확인합니다.

내 역할 웹 화면·API·Proxmox 연동과 작업 이력 저장을 맡았습니다.

백업·복원·템플릿 제작·정지 VM 이동도 구현했으며, 이 기능들의 실환경 변경 검증은 남아 있습니다.

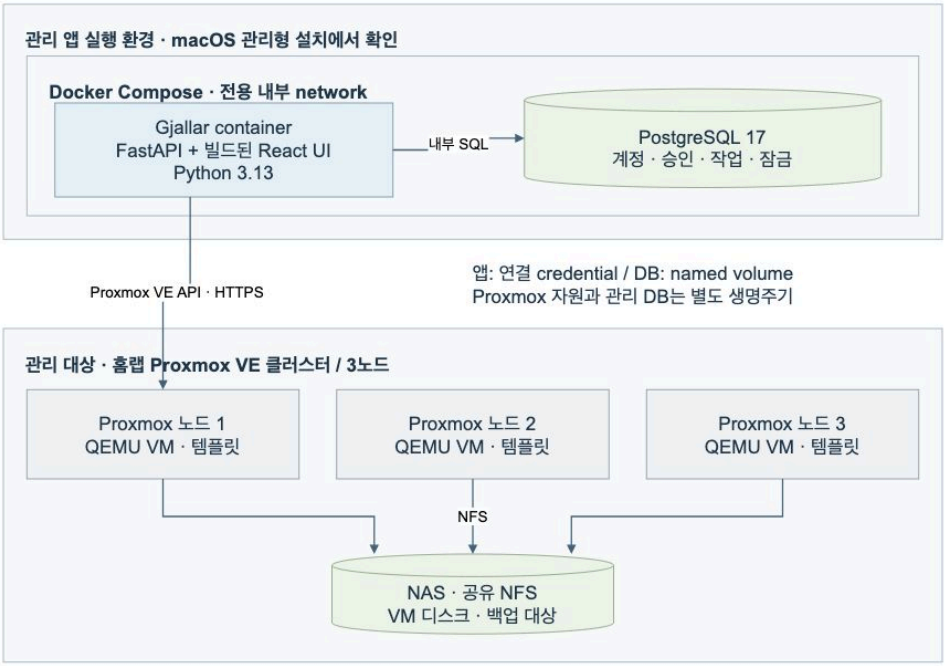
React · FastAPI · PostgreSQL · Proxmox VE API

코드·사용 가이드

Architecture

관리 도구와 Proxmox 연동

Gjallar는 변경 요청과 처리 이력을 관리하고, VM 실행과 실제 상태는 Proxmox가 담당합니다. 관리 앱은 별도로 운영 중인 홈랩의 Proxmox 3노드에 API로 연결합니다.



관리형 설치의 앱·DB와 홈랩 환경. NAS는 공유 NFS를 제공하며 개별 VM 배치는 생략했습니다.

기술 선택

필요한 기능을 Proxmox API로

다른 환경에서 시연을 준비하며 Terraform import와 Ansible SSH 권한 준비를 조사했습니다. API로 필요한 조회·변경을 구현할 수 있어 두 도구를 함께 준비하는 구성은 목적에 비해 과하다고 판단했습니다.

완료 판단

요청 응답과 실제 상태 대조

시작 요청 뒤에는 실제 VM이 실행 중인지, 사양 변경 뒤에는 설정이 반영됐는지 다시 확인합니다. 요청 식별자를 기준으로 중복 실행을 통제하고, 결과가 불확실하면 기록을 남겨 재확인합니다.

생성 전 조건 확인

초기 생성 기능에서 200GB 템플릿을 50GB로 요청해 복제 후 실패했습니다. 실제 템플릿 크기를 먼저 비교하도록 바꿨고, 50GB 요청 차단과 200GB 생성 성공을 확인했습니다. 현재 API 경로에도 이 검사를 유지했습니다. 개선 기록

작업 처리 구조 · 홈랩 실행 기록

In Use

내 홈랩을 관리하는 콘솔

● **개인 홈랩에서 사용 중** 로컬 웹 앱 · Proxmox 연결

로컬에서 Gjallar를 실행해 개인 홈랩의 Proxmox 자원을 조회하고 관리하고 있습니다. 홈랩에서 기능을 확인하고, 실제 사용 환경에서 발견한 불편을 고치며 개발을 이어 가고 있습니다.

관리 환경	조회 대상	사용 형태
Proxmox 3노드	**VM 29개**	**개인 운영 도구**

2026.09.27 홈랩 조회 화면 기준. 실제 화면은 프로젝트 개요에 수록했습니다.

01 / 일상 사용

VM 생성 준비

자주 쓰는 CPU·메모리 사양을 재사용하고 생성 전 자원 조건을 확인합니다. VM마다 사양을 처음부터 정하고 IP를 일일이 확인하던 수고가 줄었습니다.

02 / AI 활용

Hermes의 추천 사양으로 VM 생성 진행

개인 AI 에이전트 Hermes에 Gjallar 관리 권한을 위임했습니다. 내부 서비스에 필요한 VM을 준비할 때 Hermes가 추천한 사양을 바탕으로 Gjallar를 통해 생성을 진행했습니다.

03 / 사용 중 개선

실제 접속 환경에서 발견한 화면 오류 수정

홈랩 VM의 HTTP 주소에서 생성 화면이 하얗게 표시되는 문제가 있었습니다. 해당 환경에서 요청 ID 생성 기능을 사용할 수 없는 것을 확인해 공통 생성기로 바꾸고, 실제 VM에 적용한 뒤 화면과 새로고침이 정상 동작하는 것을 확인했습니다.

홈랩에서 확인한 범위

2026.09.25에는 시험 VM의 생성·부팅·사양 변경·삭제를 표준 관리 API로 확인했습니다. 백업·복원·템플릿 제작·노드 이동은 구현 후 실환경 변경 검증을 남겨두고 있습니다.

사용 환경의 개선·검증 기록 · 프로젝트 저장소